Laboration 10 – klasser, bank

Mål: Få mer träning i att implementera större program med flera klasser samt klasser med flera nyckel-värdetabeller.

Bakgrund

Uppgiften är att skriva ett program som håller reda på bankkonton och kunder i en bank. Programmet är helt textbaserat: en meny skrivs ut till konsolen och menyvalen görs via tangentbordet. Du ska skriva programmet helt själv, och det ska följa de objektorienterade principer du lärt dig i kursen.

Redovisning

För att få redovisa måste du ha kört igenom alla testfallen som är sist i detta dokument. Checka också av att du uppfyller alla krav som är listade i nästa avsnitt nedan. Exempelkörningarna som är efter uppgiftsbeskrivningen men innan testfallen innehåller också en del tips/"krav" och kan hjälpa dig mycket.

Observera att alla laborationer måste vara redovisade innan tentan. När du kommer till redovisning ska du minnas vad du gjort, dröj därför inte för länge mellan att du gör laborationen och att du redovisar den. Har du ändå glömt så läs på innan.

Krav

Kraven för programmet beskrivs nedan. Sist i häftet återfinns också ett antal exempelkörningar, som illustrerar hur ditt program ska fungera för varje menyval, samt testfall *som måste köras innan du redovisar för din labhandledare*.

För att ditt program ska bli **godkänt** ska det uppfylla följande krav:

- Programmet ska ha en meny som innehåller följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
  10. Avsluta programmet
- Klasserna `Customer`, `Account` och `Bank` ska exakt följa specifikationerna som anges under avsnittet Design nedan. Det är tillåtet att lägga till privata hjälpmetoder. (Det är inte ovanligt att en klass, för att fungera ihop med andra klasser, måste se ut på ett visst sätt – följa en viss specifikation – trots att man implementerar funktionaliteten själv.)
- Koden som startar programmet ska ligga i en egen fil/modul som ska heta `application.py`. Detta kommer vara huvudprogrammet, och ska ansvara för att skapa banken, skriva ut menyn, låta användaren göra val, skriva ut resultaten etc.

- Inga utskrifter eller inläsningar (`print()` eller `input()`) får lov att finnas i klasserna `Customer`, `Account` eller `Bank`. Allt som gäller användargränssnitt och interaktion med användaren skall istället skötas från huvudprogrammet i modulen `application.py`. Där får du lov att använda valfritt antal hjälpmetoder eller andra hjälpklasser för att sköta användargränssnittet. (Det är vanligt att man begränsar användargränssnittet till vissa moduler på detta sätt. På så sätt kan klasserna `Customer`, `Account` och `Bank` återanvändas i ett annat program med ett annat användargränssnitt.)
- Klassen `Bank` ska ha två attribut som är nyckel-värdetabeller – en för att hålla reda på alla bankkonton samt en annan för att hålla reda på alla kunder.
- Klasserna `Customer` och `Account` har ”egenskaper” i sina specifikationer nedan, som ska vara publika attribut. Alla övriga attribut som läggs till (därför att de behövs för att implementera klassernas funktionalitet) ska vara privata. Klassen `Bank` får inte ha några publika attribut alls.
- Observera metodernas returtyper (som anges i specifikationerna) och försök följa dessa.
- Kundnummer ska börja på bokstaven *C* och ska vara ett löpnummer som startar på 10 (dvs. första kunden ska få kundnummer C10).
- Kontonummer ska vara ett löpnummer som startar på 1000 (första kontot ska få kontonummer 1000).
- De indata som ges i exemplet nedan ska ge väsentligen samma utskrift som exemplen i bilagan. Du får gärna byta ordval, men ditt program ska i allt väsentligt fungera likadant och vara lika tydligt (eller tydligare) för användaren.
- Objekt i sekvens (t.ex. listor) ska skrivas ut genom att anropa print för varje ingående objekt (eller på annat sätt bygga en anpassad utskrift). Det är inte tillräckligt att printa hela listan direkt, eftersom utskriften då inte får ett tillräckligt användarvänligt format.
- Programmet ska inte krascha om användaren matar in kontonummer eller menyval som inte finns, eller indata av fel typ (exempelvis att användaren matar in bokstäver då programmet förväntar sig ett heltal).

Design

```
class Customer:

    Klass som representerar en kund

    Egenskaper:
    customer_id: str
        Kundens unika kundnummer i banken
    personal_nbr: str
        Kundens personnummer
    name: str
        Kundens namn

    Konstruktor:
    Customer(customer_id: str, personal_nbr: str, name: str):
        Skapar en kund med kundnummer customer_id,
        personnummer personal_nbr och namnet name

    Metoder:
    __str__() -> str:
        Returnerar en sträng för att användarvänligt beskriva
        denna kund, exempelvis på formen
        'Kund: C10, Namn (personnummer): Ada Lovelace (18151210)'
```

```
class Account:

    Klass som representerar ett bankkonto

    Egenskaper:
    customer_id: str
        Unikt id för kunden som äger kontot
    account_nbr: int
        Det, för banken, unika kontonummer som identifierar kontot

    Konstruktor:
    Account(customer_id: str, account_nbr: int):
        Skapar ett konto för kunden med id customer_id och
        med kontonumret account_nbr

    Metoder:
    deposit(amount: float) -> bool:
        Sätter in beloppet amount på detta konto.
        Returnerar True om insättningen lyckades och False annars
        (om beloppet är negativt)

    withdraw(amount: float) -> bool:
        Tar ut beloppet amount från detta konto.
        Returnerar True om uttaget lyckades och False annars
        (om beloppet är felaktigt eller för stort)

    current_balance() -> float:
        Returnerar det aktuella saldot på kontot.

    __str__() -> str:
        Returnerar en strängrepresentation av detta konto på formen
        'kontonummer: 1000, saldo: 0.0 (C10)'
        Där C10 är exempel på kontoinnehavarens kundnummer
```

```
class Bank:

    Klass som representerar en bank och som ger möjlighet
    att lagra konton och kunder

    Konstruktor:
    Bank():
        Skapar en bank utan konton och kunder

    Metoder:
    add_customer(name: str, personal_nbr: str) -> str | None:
        Skapar en ny kund och lägger till den till banken.
        Returnerar den nya kundens kundnummer eller None om personnumret
        redan fanns och kunden därför inte kunde skapas.

    get_customer(customer_id: str) -> Customer | None:
        Returnerar kunden med kundnummer customer_id,
        returnerar None om ingen sådan kund existerar.

    find_customer_by_part_of_name(name_part: str) -> list[Customer]:
        Returnerar en lista med de kundobjekt vars namn helt eller delvis
        innehåller strängen name_part

    create_account(customer_id: str) -> int:
        Skapar ett nytt konto åt kunden med kundnummer customer_id.
        Returnerar det nya kontots kontonummer eller -1 om kunden inte existerar.

    get_account(account_nbr: int) -> Account | None:
        Returnerar det Account-objekt som har nummer account_nbr,
        returnerar None om ett sådant konto inte kan hittas.

    remove_account(account_nbr: int) -> bool:
        Tar bort kontot med kontonummer account_nbr
        Returnerar True om kontot kunde tas bort, False annars
        (d.v.s. om kontot inte hittades eller saldot inte var noll).

    transfer(from_acc_nbr: int, to_acc_nbr: int, amount: float) -> bool:
        För över summan amount från kontot med nummer from_acc_nbr
        till kontot med nummer to_acc_nbr. Returnerar True om båda
        kontona existerar och överföringen lyckas, annars False.

    all_accounts() -> list[Account]:
        Returnerar en lista med bankens samtliga bankkonton.

    accounts_by_customer(customer_id: str) -> list[Account] | None:
        Returnerar en lista med samtliga konton som innehas av kunden
        med kundnummer customer_id.
        Returnerar None om ingen sådan kund existerar

    all_customers_sorted_by_name() -> list[Customer]:
        Returnerar en lista med bankens samtliga kunder,
        listan som returneras ska vara sorterad stigande,
        i bokstavsordning, med avseende på kundernas namn
```

Uppgifter

1. Skapa en ny katalog på översta nivån i din labbkatalog i VSCode, så att du får en ny katalog för Lab10 som ligger jämte de andra laborationerna.

2. Skapa en fil *application.py* som ska innehålla programmet som driver banken. Här ska du placera samtliga utskrifter, all inläsning samt menyval etc. Börja med att göra ett kodskelett där du redan nu kan skriva ut menyn, välja ett alternativ samt gå till ett visst val baserat på det valda alternativet. Lägg till en loop så att programmet klarar av att göra detta tills användaren avslutar. Men för att ditt program verkligen ska kunna göra något måste resten av klasserna implementeras, gå därför vidare till nästa uppgift när du har den initiala strukturen för ditt huvudprogram.

3. Skapa en fil *data.py* som ska innehålla klasserna `Customer`, `Account` och `Bank`

4. Implementera klassen `Customer` enligt specfikationen ovan.

5. Implementera klassen `Account` enligt specifikationen ovan.

6. Implementera därefter klassen `Bank` i samma fil, enligt specifikationen ovan samt anvisningarna nedan. Lös uppgiften stegvis och testa att varje metod fungerar genom att stegvis också bygga ut din kod i *application.py*.

   Anvisningar och tips (läs alla tips innan du börjar implementera!):

   - Skapa gärna hjälpmetoder för att undvika att upprepa kod. Exempelvis kan det vara bra att skapa en eller flera hjälpmetoder för att ta emot och validera indata från användaren i *application.py*, som sköter inläsning, validering och felhantering av indata (visa fel och be användaren mata in på nytt). På så vis slipper du upprepa samma kod för inläsning och validering av t.ex. menyval, kontonummer och belopp.
   - Låt `Bank`-klassen ha två nyckel-värdetabeller som attribut: en för kunderna som går från kundnummer till kunder (`Customer`-objekt) och en för kontona som går från kontonummer till konto (`Account`-objekt). Vi väljer att använda en nyckel-värdetabell även för kontona då det tillåter snabb uppslagning givet ett kontonummer och enkel borttagning (till skillnad mot en lista).
   - Konton kopplas ihop med kunder på *ett* sätt, och det är genom att varje kontoobjekt vet vilket kundnummer som är innehavare. Alltså, varje konto håller reda på sin egen ägare. Inga andra datastrukturer behövs för att göra denna koppling (alla uppgifter går att lösa ändå).
   - `Bank`-klassen ska automatiskt skapa kundnummer och kontonummer enligt kraven ovan. För att hålla reda på nästa kundnummer och nästa kontonummer kan man använda två räknare, alltså två heltal som attribut. Ett kundnummer ska vara en sträng som följer formen `C10`, `C11`, osv, så metoden `add_customer` behöver omvandla heltalet till en sträng (och lägga till bokstaven `C` framför). Kontonumret kan vara ett heltal (då ingen bokstav ska ingå).
   - Börja t.ex. med att implementera `bank.add_customer` för att därefter implementera menyval 1 i *application.py*. För att kontrollera resultatet bör du därefter implementera menyval 8 (sök efter kund) i *application.py*, vilket i sin tur kräver att metoden `find_customer_by_part_of_name` implementeras i `Bank` etc. På så vis kan du bygga upp ditt program stegvis och hela tiden testa samtidigt.

- Menyval 9 ska skriva ut alla kunder i bokstavsordning med avseende på namn. För att sortera kunderna kan du använda Pythons inbyggda sorteringsfunktionalitet, t.ex. genom att använda funktionen `sorted` på en lista med kunder. För att Python ska veta hur kunderna ska sorteras ska klassen `Customer` implementera de speciella jämförelsemetoderna ”less than”, ”greater than” och ”equal to”. Du kan implementera alla tre, men som minimum måste du implementera antingen `__lt__` *och* `__eq__`, eller `__gt__`.
- Det kan vara jobbigt att testa menyvalen om och om igen eftersom kunder och konton måste läggas in varje gång programmet startas. Under en del av testningen kan det därför vara idé att tillfälligt hårdkoda några konton och kunder som banken alltid startar med. Det är dock viktigt att dessa tas bort inför sluttestningen enligt testfallen nedan (programmet måste fungera även vid start med helt tom bank). Alternativt kan man också spara kunder på fil, enligt den frivilliga uppgiften längre ned.

7. När du börjar bli klar: Se till att samtliga meny-val har blivit implementerade i *application.py* och att alla metoder i `Bank` är färdiga. Även om du testat metoderna under tiden måste du nu testa ditt program som helhet. Funkar alla menyval tillsammans med varandra, i vilken ordning som helst? Glöm inte att kolla varje anvisning och exempel för varje menyval i avsnittet med exempelkörningarna nedan.

8. När du känner dig nöjd, sluttesta ditt program mot testfallen allra sist i denna labhandledning innan du skriver upp dig på redovisning.

Frivilliga extrauppgifter

Vill du utmana dig själv och öva mer får du gärna göra några av nedan extrauppgfter, men det är inte något krav för att bli godkänd.

1. *Frivillig extrauppgift:* Lägg till filläsning och filskrivning i ditt program. T.ex. att banken automatiskt sparar alla konton och kunder till fil innan programmet stängs av samt att programmet automatiskt läser in alla konton och kunder från filen när det startar upp igen.

2. *Frivillig extrauppgift:* Formatera utskrifter i raka kolumner. Du kan använda de inbyggda `str`-metoderna `ljust`, `rjust` och `center` för att vänster-, höger- och centerjustera text. Man kan också använda formatteringsregler i f-strängar. Prova t.ex. nedanstående kod:

```
print("|", "random string".ljust(30), "|")
print("|", "random string".rjust(30), "|")
print("|", "random string".center(30), "|")

s = "random string"
print(f"| {s:<30} |")
print(f"| {s:>30} |")
```

Nedan är ett exempel där högerjustering används för kontonummer och saldovärden:

```
7
Du har valt att skriva ut alla konton
kontonummer:  1000, saldo: 5230000.0 (C11)
kontonummer:  1001, saldo:       0.0 (C11)
kontonummer:  1002, saldo:     490.0 (C10)
kontonummer: 11273, saldo:     622.5 (C217)
kontonummer: 15661, saldo:   79508.5 (C217)
```

Exempelkörning samt anvisningar för respektive menyval

Det användaren matar in i exempelkörningarna är markerat med fetstil. Resten skrivs ut av programmet. Observera att exempelkörningarna nedan inte täcker alla fall utan är mest till för att illustrera programmets grundtanke. Se testfallen nedan för något mer heltäckande fall.

1. Skapa en ny kund

Anvisningar:

- Kunder kan heta vad som helst, och deras namn behöver inte vara unika.
- Det ska inte gå att skapa en kund med samma personnummer som en befintlig kund.
- Som ni säkert noterat går det inte att ta bort kunder, det är helt i sin ordning.

Exempel

```
Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
1
Du har valt att skapa en ny kund
Skriv in kundens namn Grace Hopper
Skriv in kundens personnummer 19061209
Kunden tillagd med kundnummer C10
```

```
Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
1
Du har valt att skapa en ny kund
Skriv in kundens namn Ada Lovelace
Skriv in kundens personnummer 18151210
Kunden tillagd med kundnummer C11
```

2. Skapa ett nytt konto

Anvisningar:

- Kunden måste redan finnas om kontot ska kunna skapas, om kunden inte finns måste användaren meddelas
- Det är viktigt att varje konto blir kopplat till rätt kund

Exempel

```
Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
2
Du har valt att skapa ett nytt konto
Ange kundnummer för kunden som ska ha kontot C11
Kontot skapades med kontonummer 1000

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
2
Du har valt att skapa ett nytt konto
Ange kundnummer för kunden som ska ha kontot C10
Kontot skapades med kontonummer 1001

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
7
Du har valt att skriva ut alla konton
kontonummer: 1000, saldo: 0.0 (C11)
kontonummer: 1001, saldo: 0.0 (C10)
```

3. Ta bort konto

Anvisningar:

- Om kontot inte finns måste användaren få veta det
- Du behöver inte rensa i kundregistret, om en kund i och med borttagandet av kontot blir helt utan konton så är det helt i sin ordning
- Om det finns pengar kvar på kontot ska det inte gå att ta bort

Exempel

```
Exemplet kör när följande konton finns i banken:
kontonummer: 1000, saldo: 0.0 (C11)
kontonummer: 1001, saldo: 0.0 (C10)

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
3
Du har valt att ta bort ett konto
Ange kontonummer för kontot som ska tas bort 3000
Kontot kunde inte tas bort

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
3
Du har valt att ta bort ett konto
Ange kontonummer för kontot som ska tas bort 1000
Kontot togs bort

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
7
Du har valt att skriva ut alla konton
kontonummer: 1001, saldo: 0.0 (C10)
```

4. Sätta in pengar

Anvisningar:

- Det ska inte gå att sätta in ett negativt belopp på kontot

Exempel

```
Exemplet kör när följande kunder och konton finns i banken:
Kund: C11 Namn (personnummer): Ada Lovelace (18151210)
       0: kontonummer: 1000, saldo: 0.0 (C11)
       1: kontonummer: 1001, saldo: 0.0 (C11)
Kund: C10 Namn (personnummer): Grace Hopper (19061209)
       0: kontonummer: 1002, saldo: 0.0 (C10)

Välj ett av följande alternativ:

  1. Skapa en ny kund

  2. Skapa ett nytt konto

  3. Ta bort konto

  4. Sätta in pengar

  5. Ta ut pengar

  6. Överför pengar

  7. Skriv ut alla konton

  8. Sök på kund på del av namn

  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton

 10. Avsluta programmet

 4
Du har valt att sätta in pengar
Ange kontonummer 1002
Ange belopp 1050
Insättning lyckades, saldot på kontot är nu 1050.0

Välj ett av följande alternativ:

  1. Skapa en ny kund

  2. Skapa ett nytt konto

  3. Ta bort konto

  4. Sätta in pengar

  5. Ta ut pengar

  6. Överför pengar

  7. Skriv ut alla konton

  8. Sök på kund på del av namn

  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton

 10. Avsluta programmet

4
Du har valt att sätta in pengar
Ange kontonummer 1009
Ange belopp 300
Kontot hittades inte

Välj ett av följande alternativ:

  1. Skapa en ny kund

  2. Skapa ett nytt konto

  3. Ta bort konto

  4. Sätta in pengar

  5. Ta ut pengar

  6. Överför pengar

  7. Skriv ut alla konton

  8. Sök på kund på del av namn

  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
```

```
 10. Avsluta programmet
9
Du har valt att skriva ut alla kunder tillsammans med deras konton
Kund: C11 Namn (personnummer): Ada Lovelace (18151210)
       0: kontonummer: 1000, saldo: 0.0 (C11)
       1: kontonummer: 1001, saldo: 0.0 (C11)
Kund: C10 Namn (personnummer): Grace Hopper (19061209)
       0: kontonummer: 1002, saldo: 1050.0 (C10)
```

5. Ta ut pengar

Anvisningar:

- Det ska inte gå att ta ut pengar om saldot på kontot inte är tillräckligt

Exempel

```
Följande kunder och konton finns när exemplet kör
Kund: C11 Namn (personnummer): Ada Lovelace (18151210)
       0: kontonummer: 1000, saldo: 0.0 (C11)
       1: kontonummer: 1001, saldo: 0.0 (C11)
Kund: C10 Namn (personnummer): Grace Hopper (19061209)
       0: kontonummer: 1002, saldo: 1050.0 (C10)

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
5
Du har valt att ta ut pengar
Ange kontonummer 1002
Ange belopp 60
Uttag lyckades, saldot på kontot är nu 990.0

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
5
Du har valt att ta ut pengar
Ange kontonummer 1001
Ange belopp 250
Uttag misslyckades, saldot på kontot är 0.0

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
```

```
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
7
Du har valt att skriva ut alla konton
kontonummer: 1000, saldo: 0.0 (C11)
kontonummer: 1001, saldo: 0.0 (C11)
kontonummer: 1002, saldo: 990.0 (C10)
```

6. Överför pengar

Anvisningar:

- Ingen förändring ska ske förrän båda kontona hittats och man är säker på att det finns tillräckligt med pengar
- Om något av kontona inte hittats eller pengarna inte räcker ska användaren få ett felmeddelande, men för att bli godkänd är det ok att bara meddela *"Överföring misslyckats"*, utan att specificera anledningen

Exempel

```
Följande konton finns när exemplet körs
kontonummer: 1000, saldo: 0.0 (C11)
kontonummer: 1001, saldo: 0.0 (C11)
kontonummer: 1002, saldo: 990.0 (C10)

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
6
Du har valt att överföra pengar
Ange kontonummer att överföra från 1002
Ange kontonummer att överföra till 1000
Ange belopp 500
Överföring lyckades

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
```

```
  8. Sök på kund på del av namn

  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton

 10. Avsluta programmet
7
Du har valt att skriva ut alla konton
kontonummer: 1000, saldo: 500.0 (C11)
kontonummer: 1001, saldo: 0.0 (C11)
kontonummer: 1002, saldo: 490.0 (C10)
```

7. Skriv ut alla konton

Anvisningar:

- Utskriften ska visa kontoinformation, det behöver framgå vilket kundnummer som är innehavare av kontot men ingen annan kundinformation behövs

Exempel
Detta menyalternativ används bl.a. i exemplen för alternativ 5 och 6

8. Sök på kund på del av namn

Anvisningar:

- Sökningen ska vara okänslig för små och stora bokstäver (en sökning på *Aa* ska alltså även matcha *aa*)

Exempel

```
Exemplet kör när följande kunder och konton finns i banken:
Kund: C11 Namn (personnummer): Ada Lovelace (18151210)
       0: kontonummer: 1000, saldo: 0.0 (C11)
       1: kontonummer: 1001, saldo: 0.0 (C11)
Kund: C10 Namn (personnummer): Grace Hopper (19061209)
       0: kontonummer: 1002, saldo: 0.0 (C10)

Välj ett av följande alternativ:
  1. Skapa en ny kund

  2. Skapa ett nytt konto

  3. Ta bort konto

  4. Sätta in pengar

  5. Ta ut pengar

  6. Överför pengar

  7. Skriv ut alla konton

  8. Sök på kund på del av namn

  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton

 10. Avsluta programmet
8
Du har valt att söka på (del av) kundnamn
Skriv in (del av) namn att söka på: Grace
Följande kunder hittades:
Kund: C10 Namn (personnummer): Grace Hopper (19061209)

Välj ett av följande alternativ:
  1. Skapa en ny kund

  2. Skapa ett nytt konto

  3. Ta bort konto

  4. Sätta in pengar

  5. Ta ut pengar

  6. Överför pengar

  7. Skriv ut alla konton
```

```
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
8
Du har valt att söka på (del av) kundnamn
Skriv in (del av) namn att söka på: LOVE
Följande kunder hittades:
Kund: C11 Namn (personnummer): Ada Lovelace (18151210)

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
8
Du har valt att söka på (del av) kundnamn
Skriv in (del av) namn att söka på: ace
Följande kunder hittades:
Kund: C10 Namn (personnummer): Grace Hopper (19061209)
Kund: C11 Namn (personnummer): Ada Lovelace (18151210)
```

9. Skriv ut samtliga kunder (bokstavsordning) och dess konton

Anvisningar:

- Kunderna ska vara sorterade i bokstavsordning
- Under varje kund ska kundens konto visas (om det finns några)
- Listan över kundens konton behöver inte vara sorterad
- Om det inte finns några kunder alls ska användaren få ett meddelande om det

Exempel
Detta ser du exempel på i körningen för menyalternativ nr 4 (Sätta in pengar)

10. Avsluta programmet

Anvisningar:

- Användaren ska få meddelande om att programmet avslutats

Exempel

```
 Välj ett av följande alternativ:

  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
10
Du har valt att avsluta programmet, välkommen tillbaka
```

Felaktigt menyval

Anvisningar:

- Efter att användaren meddelats att valet inte finns ska menyn visas igen

Exempel

```
Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
22
Felaktigt val, välj ett alternativ mellan 1 och 10

Välj ett av följande alternativ:
  1. Skapa en ny kund
  2. Skapa ett nytt konto
  3. Ta bort konto
  4. Sätta in pengar
  5. Ta ut pengar
  6. Överför pengar
  7. Skriv ut alla konton
  8. Sök på kund på del av namn
  9. Skriv ut samtliga kunder (bokstavsordning) och dess konton
 10. Avsluta programmet
```

Testning och testfall för sluttest - bankuppgiften

Läs igenom testfallen i ett tidigt stadium. Testfallen är gjorda för att kunna följas från början till slut. Men: Glöm inte att under utvecklingen hela tiden testa ditt program fritt efter eget huvud och med egenpåhittad data. Kör därefter testfallen efter att hela ditt program fungerar hyfsat, det är nämligen inte säkert att du bäst implementerar funktionerna i den ordning som testfallen har. Testfallen specificerar vilka situationer ditt program ska klara av, det hindrar inte din labhandledare från att testa situationerna på annat sätt, t.ex. i en annan ordning eller med annan data.

Observera att testfallen kräver att kunderna får ett kundnummer på formen Cxx där xx är 10 för allra första kunden, 11 för nästa etc.

Testfall 1: Lista alla konton när inget konto finns i banken

- Notera att detta testfall ska köras på en bank utan konton

1. Välj menyalternativ 7 (Skriv ut alla konton)
2. Kontrollera att programmet inte kraschar
3. Kontrollera att användaren får en vänlig text om att det inte finns några konton
4. Kontrollera att det går att göra ett nytt menyval (t.ex. genom att påbörja testfall 2 direkt)

Testfall 2: Lista alla kunder när inga kunder finns i banken

- Notera att detta testfall ska köras på en bank utan konton

1. Välj menyalternativ 9 (Skriv ut samtliga kunder (bokstavsordning) och dess konton)
2. Kontrollera att programmet inte kraschar
3. Kontrollera att användaren får en vänlig text om att det inte finns några kunder
4. Kontrollera att det går att göra ett nytt menyval (t.ex. genom att påbörja testfall 3 direkt)

Testfall 3: Lägg till en ny kund

1. Välj menyalternativ 1 (Skapa en ny kund)
2. Ange kontoinformation enligt följande [namn: Anna B, personnummer: 1234] när programmet efterfrågar det
3. Kontrollera att du får veta vilket kundnummer den nya kunden fått
4. Kontrollera att du kommer tillbaka till menyn efter att kontot skapats
5. Välj menyalternativ 9 (Skriv ut samtliga kunder (bokstavsordning) och dess konton)
6. Kontrollera att kunden du nyss skapade finns med
7. Kontrollera att informationen om kunden är korrekt
8. Kontrollera att all information visas samt är tydlig.
9. Kontrollera att du kommer tillbaka till menyn och kan göra ett nytt menyval

Testfall 4: Lägg till en kund med samma personnummer

1. Välj menyalternativ 1 (Skapa en ny kund)
2. Ange kontoinformation enligt följande [namn: Sara, personnummer: 1234] när programmet efterfrågar det
3. Kontrollera att du får veta att det redan finns en kund med detta personnummer
4. Kontrollera att du kommer tillbaka till menyn
5. Välj menyalternativ 9 (Skriv ut samtliga kunder (bokstavsordning) och dess konton)
6. Kontrollera att endast kunden från testfall 3 finns med i utskriften (d.v.s. att ingen kund skapades i detta testfall)
7. Kontrollera att du kommer tillbaka till menyn och kan göra ett nytt menyval

Testfall 5: Lägg till en kund med samma namn

1. Välj menyalternativ 1 (Skapa en ny kund)
2. Ange kontoinformation enligt följande [namn: Anna B, personnummer: 1717] när programmet efterfrågar det
3. Kontrollera att du får veta vilket kundnummer den nya kunden fått
4. Kontrollera att du kommer tillbaks till menyn
5. Välj menyalternativ 9 (Skriv ut samtliga kunder (bokstavsordning) och dess konton)
6. Kontrollera att båda Anna B (fast med olika personnummer) finns med
7. Kontrollera att du kommer tillbaks till menyn och kan göra ett nytt menyval

Testfall 6: Skapa konto

1. Välj menyalternativ 2 (Skapa ett nytt konto)
2. Ange kontoinformation enligt följande [kundnummer: C10] när programmet efterfrågar det
3. Kontrollera att du får veta vilket kontonummer det nya kontot fått (bör vara 1000 om anvisningarna följts)
4. Kontrollera att du kommer tillbaks till menyn efter att kontot skapats
5. Välj alternativ 7
6. Kontrollera att kontot du nyss skapade finns med
7. Kontrollera att inga andra konton än ovan nämnda finns med
8. Kontrollera att du kommer tillbaks till menyn och kan göra ett nytt menyval

Testfall 7: Skapa konto för kund som inte finns

1. Välj menyalternativ 2 (Skapa ett nytt konto)
2. Ange ett kundnummer som inte finns (t.ex. 111) när programmet efterfrågar det
3. Kontrollera att du får veta att ingen sådan kund finns
4. Kontrollera att du kommer tillbaka till menyn efter att kontot skapats
5. Välj alternativ 7
6. Kontrollera att listan över konton är oförändrad
7. Kontrollera att du kommer tillbaka till menyn och kan göra ett nytt menyval

Testfall 8: Sorterad lista

1. Lägg till följande kunder (via menyval 1): [namn: Jesper Z, personnummer: 1313] [namn: Anders A, personnummer: 1212],
2. Välj menyalternativ 9 (Skriv ut samtliga kunder (bokstavsordning) och dess konton)
3. Kontrollera att namnen i kontoutskriften kommer i bokstavsordning (Anders A, Anna B, Jesper Z). Den inbördes ordningen mellan de olika Anna B är valfri
4. Kontrollera att kontot för kund C10 (Anna B, 1234) finns med i listningen och att det tydligt framgår att kontot tillhör henne

Testfall 9: Sorterad lista av kunder, nu med flera konton

1. Lägg till ett konto för C10 (Anna B) ett för C12 (Jesper Z) samt ett konto för C13 (Anders A)
2. Välj menyalternativ 9 (Skriv ut samtliga kunder (bokstavsordning) och dess konton)
3. Kontrollera att listningen är korrekt och att samtliga fyra konton visas under rätt kund
4. Kontrollera att du kommer tillbaka till menyn och kan göra ett nytt menyval

Testfall 10: Sök på del av namn (case-insensitive, en träff)

1. Välj menyalternativ 8 (Sök på kontoinnehavares (del av) namn)
2. När programmet frågar efter söksträngen: mata in jesper
3. Kontrollera att utskriften ger dig information om kunden Jesper Z (skapad i testfall 5)
4. Kontrollera att inga andra kunder är med i utskriften

Testfall 11: Sök på del av namn (många träffar)

1. Välj menyalternativ 8 (Sök på kontoinnehavares (del av) namn)
2. När programmet frågar efter söksträngen: mata in *An*
3. Kontrollera att båda Anna B samt Anders A dyker upp i listningen

Testfall 12: Sök på del av namn (ej början av strängen)

1. Välj menyalternativ 8 (Sök på kontoinnehavares (del av) namn)
2. När programmet frågar efter söksträngen: mata in *ders*
3. Kontrollera att Anders A dyker upp i listningen
4. Kontrollera att inga andra kunder är med i listan

Testfall 13: Sätta in pengar, normalfallet

1. Välj menyalternativ 9
2. Notera kontonumret som Jesper Zs konto har
3. Välj menyalternativ 4 (Sätt in pengar)
4. När programmet frågar efter kontonummer: ange numret som du hittade i steg 2
5. När programmet frågar efter belopp: ange 500
6. Välj menyalternativ 7 (skriv ut alla konton)
7. Kontrollera kontot för Jesper Z, saldot ska vara 500

Testfall 14: Sätta in pengar, negativt belopp

1. Välj menyalternativ 4 (Sätt in pengar)
2. När programmet frågar efter kontonummer: ange det kontonummer du använde i testfall 13
3. När programmet frågar efter belopp: ange -1000
4. Kontrollera att programmet meddelar att negativa belopp ej är tillåtna
8. Välj menyalternativ 7 (skriv ut alla konton)
5. Kontrollera kontot för Jesper Z, saldot ska vara 500

Testfall 15: Sätta in pengar, kontrollera att det blir rätt konto

1. Välj menyalternativ 9
2. Notera de kontonummer som Anna Bs båda konton har
3. Välj menyalternativ 4 (Sätt in pengar)
4. När programmet frågar efter kontonummer: ange ett av de nummer som du hittade i steg 2
5. När programmet frågar efter belopp: ange 5000
9. Välj menyalternativ 7 (skriv ut alla konton)
6. Kontrollera kontona för Anna B, kontrollera att saldot är 5000 på det konto du valde i steg 4.
7. Kontrollera att saldot är noll på Anna Bs andra konto

Testfall 16: Sätta in pengar, kontot finns inte

1. Välj menyalternativ 4 (Sätt in pengar)
2. När programmet frågar efter kontonummer: ange ett nummer som du vet inte finns i banken
3. Om programmet frågar efter belopp: ange 500
4. Kontrollera att du får meddelande om att kontot ej existerar
5. Kontrollera att programmet inte kraschar
6. Kontrollera att du kommer tillbaka till menyn och kan göra ett nytt menyval
7. Välj menyalternativ 7
8. Kontrollera så att samtliga konton har oförändrat saldo

(notera: när kontot inte finns är det valfritt om programmet frågar efter belopp eller ej)

Testfall 17: Ta ut pengar, normalfallet

1. Välj menyalternativ 9
2. Notera kontonumret som Jesper Zs konto har
3. Välj menyalternativ 5 (Ta ut pengar)
4. När programmet frågar efter kontonummer: ange numret som du hittade i steg 2
5. När programmet frågar efter belopp: ange 100
6. Välj menyalternativ 9
7. Kontrollera kontot för Jesper Z, saldot ska vara 400

Testfall 18: Ta ut pengar, kontot finns inte

1. Välj menyalternativ 5 (Ta ut pengar)
2. När programmet frågar efter kontonummer: ange ett nummer som du vet inte finns i banken
3. Om programmet frågar efter belopp: ange 500
4. Kontrollera att du får meddelande om att kontot ej existerar
5. Kontrollera att programmet inte kraschar
6. Kontrollera att du kommer tillbaka till menyn och kan göra ett nytt menyval
7. Välj menyalternativ 7 (skriv ut alla konton)
8. Kontrollera så att samtliga konton har oförändrat saldo

Testfall 19: Ta ut pengar, täckning saknas

1. Välj menyalternativ 9
2. Notera kontonumret som Anders As konto har
3. Välj menyalternativ 5 (Ta ut pengar)
4. När programmet frågar efter kontonummer: ange numret som du hittade i steg 2
5. Om programmet frågar efter belopp: ange 100
6. Kontrollera att du får meddelande om att det inte finns tillräckligt med pengar
7. Välj menyalternativ 9
8. Kontrollera kontot för Anders A, saldot ska vara noll

Testfall 20: Överföring av pengar, normalfallet

1. Välj menyalternativ 6 (Överför pengar)
2. Se till att ha noterat kontonumren för Anders A och Jesper Z (noterades i tidigare testfall)
3. När programmet frågar vilket konto pengarna ska överföras från: mata in kontonumret för Jesper Zs konto
4. När programmet frågar vilket konto pengarna ska överföras till: mata in kontonumret för Anders As konto
5. När programmet frågar efter belopp: ange 200
6. Välj menyalternativ 7
7. Kontrollera att Anders A och Jesper Z båda har saldot 200

Testfall 21: Överföring av pengar, täckning saknas

1. Välj menyalternativ 6 (Överför pengar)
2. Se till att ha noterat kontonumren för Anders A och Jesper Z (noterades i tidigare testfall)
3. När programmet frågar vilket konto pengarna ska överföras från: mata in kontonumret för Jesper Zs konto
4. När programmet frågar vilket konto pengarna ska överföras till: mata in kontonumret för Anders As konto
5. När programmet frågar efter belopp: ange 6000
6. Välj menyalternativ 7
7. Kontrollera att Anders A och Jesper Z båda har saldot 200

Testfall 22: Ta bort konto (normalfallet)

1. Välj menyalternativ 7
2. Notera ett kontonummer för ett konto som har saldot noll
3. Välj menyalternativ 3 (Ta bort konto)
4. När programmet frågar efter kontonummer: ange numret som du hittade i steg 2
5. Välj menyalternativ 7
6. Kontrollera att kontot du just tog bort inte finns i listan
7. Kontrollera att programmet inte kraschar utan återgår till menyn som vanligt

Testfall 23: Ta bort konto (kontot ej tomt)

1. Välj menyalternativ 7
2. Notera ett kontonummer för ett konto som har pengar
3. Välj menyalternativ 3 (Ta bort konto)
4. När programmet frågar efter kontonummer: ange numret som du hittade i steg 2
5. Kontrollera att programmet meddelar att kontot inte kunde tas bort (det måste inte förklara varför)
6. Välj menyalternativ 7
7. Kontrollera att kontot du just försökte ta bort finns kvar i listan
8. Kontrollera att programmet inte kraschar utan återgår till menyn som vanligt

Testfall 24: Ta bort konto (kontot finns ej)

1. Välj alternativ 7 och notera snabbt hur många konton som finns
2. Välj menyalternativ 3 (Ta bort konto)
3. När programmet frågar efter kontonummer: ange ett nummer som du är säker på inte finns i banken
4. Kontrollera att programmet inte kraschar samt meddelar att kontot inte kunde tas bort
5. Välj alternativ 7 och kontrollera att alla konton finns kvar

Testfall 25: Avsluta

1. Välj menyalternativ 10 (avsluta)
2. Kontrollera att programmet meddelar att banken nu avlutas och att programmet slutar köra (typiskt att man kommer tillbaka ut i terminalen i VSCode)

Testfallen är inte 100% heltäckande. För att verkligen hårdtesta hela systemet kan man anse att det finns vissa luckor. Fundera gärna på vilka testfall ni tycker saknas.